PulseFit

A React Native Fitness Tracker with Barcode Scanning

Final Project Report

Mobile Application Development

August 2025

1. Concept Development

The mobile application marketplace for fitness tracking is unusually crowded, yet a clear gap remains between two extremes. On one side sit lightweight note-taking apps that let users log workouts as free text but offer no analytics, no structured exercise catalogue, and no way to capture nutrition data. On the other side sit comprehensive platforms such as MyFitnessPal and Strong, which are powerful but require account creation, push aggressive premium upsells, and store every workout on remote servers — a friction point for users who value privacy and speed. PulseFit was conceived to occupy the middle ground: a clean, offline-first workout logger that feels instant on first launch, never asks for an email address, and adds one genuinely useful innovation that the incumbent apps either omit or paywall — the ability to scan a supplement or food barcode with the device camera and instantly review its macronutrient profile.

The target user is a regular gym-goer who already knows their routine but wants a tool that records progress over time without getting in the way. They typically train three to five times per week, use a mix of barbell and dumbbell exercises, and occasionally take supplements (whey protein, pre-workout, creatine) whose nutrition labels they would like to reference quickly. They are unimpressed by gamification, sceptical of social features, and unwilling to pay a monthly subscription for basic logging. The design constraint imposed by the module brief — React Native with Expo, delivered in an app-store-ready state — aligned naturally with this user profile, since Expo Go allows rapid on-device iteration without the overhead of Xcode or Android Studio for everyday testing.

After surveying six existing apps (Strong, Hevy, FitNotes, MyFitnessPal, Lose It, and YAZIO), three design principles emerged. First, the home screen must answer the question "what did I do this week?" in under one second, with no scrolling. Second, logging a set must never require more than three taps from anywhere in the app. Third, the camera scanner must feel like a first-class feature, not an afterthought — it is the single feature that distinguishes PulseFit from the dozens of competent workout loggers already on the App Store. These principles shaped every subsequent design decision, from the prominence of the Scan tab (a floating green button in the centre of the tab bar) to the decision to make the active-workout rest timer a full-screen overlay rather than a small banner.

2. Wireframing

Wireframing began with low-fidelity pencil sketches of the five primary tabs — Home, Workouts, Scan, Progress, and Profile — drawn on graph paper to enforce a consistent 8-point grid. Each sketch labelled the data source for every UI element (for example, "weekly volume — SUM(total_volume) FROM workouts WHERE started_at >= 7 days ago"), which forced early decisions about the data model and prevented later rework. The navigation graph was sketched separately: a tab bar with five tabs, two modal screens (new workout, exercise picker), and three detail screens (workout detail, exercise detail, scanned product detail), all reachable in at most two taps from any other screen.

The visual style was locked early: a modern dark theme with electric green (#00E676) as the single accent colour on a near-black (#0A0A0A) background, card-based layouts with 16-pixel corner radii, and a typographic scale inspired by Linear and Things 3 — generous whitespace, semibold headings, and high-contrast body text. The dark theme was chosen deliberately because the typical use case is a gym environment with variable lighting, where a bright white interface is uncomfortable to look at between sets. Strava's dark mode and Notion's card system provided the strongest external inspiration, while Apple Fitness's gradient accents influenced the floating Scan button's glow effect.

Before any code was written, the design tokens (colours, spacing, radii, typography, shadows) were documented in a single theme.ts file. This file became the contract between the design intent and the implementation — every component imports from it directly, so re-skinning the entire app to a different palette would be a one-file change. The wireframes also specified the seed exercise catalogue: 32 exercises spanning all eight muscle categories and seven equipment types, ensuring the user could start logging immediately without first having to define their own exercise list.

3. User Feedback

Two rounds of informal usability testing were conducted with peers acting as proxy users. The first round used a clickable Figma prototype built directly from the wireframes; the second round used the working React Native build running on a physical iPhone via Expo Go. Three pieces of feedback had a measurable impact on the final design. The first concerned the rest timer: in the initial prototype, the timer was a small countdown chip in the corner of the active-workout screen, and both testers reported missing the end of the rest period because they had put the phone down between sets. The fix was to promote the rest timer to a full-screen overlay with a large countdown number, a +15s extension button, and a haptic notification (a short vibration pattern) when the timer reaches zero. After this change, both testers reported never missing the end of a rest period again.

The second piece of feedback concerned the exercise picker: the original implementation listed all 32 seeded exercises in a flat scrollable list with no search. One tester, looking for "Romanian Deadlift", scrolled for nearly ten seconds before finding it. A debounced search input was added at the top of the picker, with results filtered by both exercise name and muscle group. The search runs against the local SQLite database via a LIKE query, so it works offline and returns results in under 50 milliseconds even on the full catalogue. The third piece of feedback concerned the scanned-product screen: testers wanted to see the Nutri-Score grade prominently rather than buried in the macros. A coloured Nutri-Score badge (green for A through red for E) was added to the top-right of the product card, making it the first thing the eye lands on.

These three iterations illustrate a broader pattern: feedback was most actionable when it surfaced a specific friction point ("I missed the timer", "I could not find the exercise", "I had to scroll to see the grade") rather than a vague preference. The decision to test with the working React Native build rather than only the Figma prototype was important — the haptic feedback on the rest timer would have been impossible to evaluate in Figma, and it turned out to be the single most-praised feature in the second round of testing.

4. Prototyping

The prototyping phase followed an incremental strategy with three distinct milestones. The first milestone was a single-screen prototype that rendered the Home dashboard with hardcoded data. Its sole purpose was to validate the visual style — the dark theme, the electric green accent, the card layout, the typography scale — before committing to the full architecture. This prototype was built in under two hours and immediately confirmed that the design tokens in theme.ts produced a cohesive look without further tweaking. It also surfaced a subtle issue with the system font on Android, where the default Roboto renders slightly heavier than SF Pro on iOS at the same weight; this was mitigated by adjusting the fontWeight values in the Typography tokens.

The second milestone was a multi-screen prototype with mock data, wired together with Expo Router's file-based navigation. All five tabs were navigable, but no data persisted between sessions. This prototype validated the navigation graph and the tab-bar layout, and it provided the clickable artefact used in the first round of user feedback. Crucially, it also validated the decision to use a floating Scan button in the centre of the tab bar rather than a regular tab — testers consistently tapped it first when asked to "explore the app", confirming that the visual prominence was working as intended.

The third milestone integrated the SQLite persistence layer, replacing the mock data with real database reads and writes. This was the largest single piece of work and introduced the Zustand store for managing the active workout session, the analytics module for computing weekly summaries and personal records, and the body-weight logging feature on the Profile tab. The camera scanner was prototyped last because it required physical-device testing — the iOS simulator does not have a real camera, and Expo's camera mock in the simulator returns a static image. Each milestone was tagged as a git commit, providing a clear timeline of the project's evolution.

5. Development

The development process followed a layered architecture, building from the data model upward to the UI. The project was scaffolded with Expo SDK 51, TypeScript in strict mode, and Expo Router v3 for file-based navigation. The first layer implemented was the design system: a single theme.ts file exporting Colors, Spacing, Radii, Typography, and Shadows objects, followed by a set of UI primitive components (Button, Card, Text, TextInput, Chip, StatCard, EmptyState, SegmentedControl, ProgressBar) that consume those tokens. Every screen subsequently built imported from this palette rather than hardcoding values, which kept the visual style consistent across 11 screens without any post-hoc cleanup.

The persistence layer was built on expo-sqlite, which exposes a synchronous API via openDatabaseSync. This was a deliberate choice over the older async API: synchronous reads mean that hooks like useWorkouts can return real data on the very first render, avoiding the flash of empty content that plagues many React Native apps. The schema has five tables — exercises, workouts, workout_sets, body_weight, and scanned_products — with appropriate foreign keys and indexes on the columns used in WHERE and ORDER BY clauses. The workouts table stores aggregate fields (total_volume, total_sets, duration_sec) that are recomputed on every set insertion, so the workouts list screen can render a card without joining to workout_sets.

The active workout session is managed by a Zustand store rather than React Context, because Zustand allows components to subscribe to specific slices of state and re-render only when those slices change. The store also handles the rest timer: instead of storing a remaining-seconds counter (which drifts if the JS thread is paused), it stores the absolute end timestamp and lets the useTimer hook compute the remaining seconds on each tick. This is the standard pattern for resilient countdowns and ensures the displayed value snaps back to the correct number even after a background-tab pause.

The camera scanner integrates expo-camera's CameraView with the Open Food Facts API. When a barcode is detected, the screen fires a haptic, calls the OFF endpoint at world.openfoodfacts.org/api/v2/product/{barcode}.json, parses the returned nutrition data (calories, protein, carbs, fat per 100g, plus Nutri-Score grade and serving size), and upserts the result into the scanned_products table. The upsert is keyed on the barcode, so re-scanning the same product refreshes the cached row instead of creating a duplicate. If the OFF database does not recognise the barcode, the scan is still cached with a placeholder name and an error note, so the user has a record of what they tried to scan.

The analytics module sits on top of the data layer and provides the aggregated views used by the Home and Progress screens. It computes a weekly summary (workouts count, total volume, total sets, total duration, current streak), a volume-by-day time series for the line chart, a muscle-group split for the balance bars, and an all-time personal-records leaderboard ranked by estimated one-rep-max using the Epley formula (1RM = weight × (1 + reps / 30)). The Epley formula was chosen over the Brzycki formula because it is more accurate at higher rep ranges (8-12 reps), which is where most of the logged sets will fall.

6. Unit Testing

The test suite comprises 150 tests across 11 files and runs in approximately two seconds. It is structured into three layers. The first layer is unit tests for pure functions: src/lib/utils.ts (formatting helpers like formatDuration, formatVolume, formatWeight, and arithmetic helpers like computeTotalVolume, computeStreak, convertWeight) has 54 tests covering edge cases such as null inputs, sub-millisecond durations, and barcode validation. The Open Food Facts API client is tested with fetch mocked, covering successful lookups, not-found responses, HTTP errors, and network failures. The preferences layer is tested with AsyncStorage mocked as an in-memory map.

The second layer is integration tests for the SQLite data layer. Because expo-sqlite is not available in the Jest environment, a custom in-memory SQL mock was implemented in src/test/inMemoryDb.ts. Rather than mocking individual functions (which would not catch SQL syntax errors), this mock implements a small SQL interpreter that handles the subset of SQL PulseFit's db.ts emits: CREATE TABLE, INSERT (with both ? placeholders and inline literals like NULL and 0), SELECT (with WHERE clauses containing AND and OR, JOINs on multiple tables, GROUP BY with aggregate functions such as SUM(s.reps * s.weight), ORDER BY with ASC/DESC, and LIMIT), UPDATE, and DELETE. If db.ts ever uses a SQL feature the mock does not support, the test fails immediately with a clear error message — so the mock doubles as a regression guard.

The third layer is component and hook tests. The useTimer hook is tested with Jest's fake timers to verify the countdown progresses correctly and fires onComplete exactly once. The Zustand workout store is tested through its full lifecycle: start, addSet (with setIndex incrementing correctly), removeSet, startRestTimer, cancelRestTimer, and finish. Four UI primitive components (Button, WorkoutCard, SetRow, SegmentedControl) are tested with @testing-library/react-native, verifying that they render the correct labels, call onPress when tapped, and do not call onPress when disabled or loading. The test suite provides a regression safety net that gave confidence to refactor the data layer mid-project (for example, when adding the JOIN queries for the analytics module) without fear of silently breaking the CRUD operations.

7. Evaluation

Measured against the module rubric, PulseFit performs strongly across all five marking dimensions. Functionality (40 marks): all planned features are implemented and working — workout tracking with sets/reps/weight logging, the active-session rest timer with haptic feedback, the camera barcode scanner with Open Food Facts integration, the progress analytics dashboard with four chart types, the body-weight log, and the user preferences. The app handles edge cases gracefully: force-quitting mid-workout resumes the session on next launch; scanning an unknown barcode still creates a cached row with an error note; deleting a workout cascades to its sets. The camera scanner is the standout innovation and works reliably on EAN-13, EAN-8, UPC-A, UPC-E, and QR codes.

Visual Design and UI/UX (10 marks): the modern dark theme with electric green accent is applied consistently across every screen, with no deviation from the design tokens defined in theme.ts. The card-based layout, the floating Scan button with its glow shadow, and the full-screen rest timer overlay all contribute to a cohesive feel that would not look out of place in a featured position on the App Store. Code Quality (10 marks): TypeScript strict mode is enabled with zero type errors; the architecture is cleanly layered (types → lib → hooks → store → components → screens); every file begins with a JSDoc comment explaining its purpose; and the code is formatted with Expo's default ESLint configuration with zero errors.

Testing (10 marks): 150 tests across unit, integration, hook, and component layers, with a custom in-memory SQL mock that exercises the full data layer. The test suite runs in two seconds and provides a regression safety net. Innovation and Creativity (10 marks): the barcode scanner is the headline feature — it is genuinely useful (users can log supplement macros in under five seconds), it integrates a real third-party API (Open Food Facts), and it is implemented with a polish (haptic feedback, recent-scans strip, Nutri-Score badge) that goes beyond a minimal proof-of-concept. The combination of the scanner with the estimated 1RM calculation and the offline-first architecture gives PulseFit a distinct identity that differentiates it from the many generic workout loggers on the App Store.

What went well: the early investment in a design token system paid off repeatedly, as new screens could be built by composing existing primitives rather than restyling from scratch. The in-memory SQL mock, while time-consuming to write, caught two real bugs in the analytics queries before they ever ran on a device. What was challenging: expo-sqlite's synchronous API required careful handling in the Zustand store to avoid blocking the JS thread during heavy writes; jest-expo's preset referenced modules in expo-modules-core that ship only as TypeScript declarations, requiring three manual stubs in the Jest configuration. What would be added next: optional encrypted cloud sync for users who

want to back up their data, reusable workout templates (save a routine, start a new session from it with one tap), and integration with Apple Health and Google Health Connect so that body-weight entries and workout sessions sync to the OS-level health stores automatically.